

Data Structures Interview Questions

Բոլոր հնարավոր DS հարցազրույցի հարցերը (C++)

Այս ֆայլը Data Structures-ի interview հարցերի ամբողջական ժողովածու է: Երեք մաս.

Մաս A — 40 multiple-choice հարց (4 տարբերակ, մեկը ճիշտ):

Մաս B — 40 բաց (open) տեսական հարց՝ առանց տարբերակների:

Մաս C — 20 tricky կոդային/complexity հարց:

Բոլոր պատասխանները **միայն վերջի էջերում** են (Answer Key A, B, C): Առաջ նայելուց առաջ փորձիր ինքդ պատասխանել:

ՄԱՍ A — Multiple Choice (40)

Q1

Array-ի element-ին index-ով մուտքը ինչ complexity է: - (a) $O(1)$ - (b) $O(n)$ - (c) $O(\log n)$ - (d) $O(n^2)$

Q2

Array-ում memory-ն ինչպես է պահվում: - (a) scattered - (b) contiguous (կոդք-կոդքի) - (c) tree-ի պես - (d) random

Q3

Linked list-ում element-ի access-ը (index-ով) ինչ է: - (a) $O(1)$ - (b) $O(n)$ - (c) $O(\log n)$ - (d) $O(1)$ average

Q4

Singly linked list-ի node-ը ինչ ունի: - (a) միայն data - (b) data + next pointer - (c) data + next + prev - (d) միայն pointer

Q5

Doubly linked list-ի node-ը ինչ ունի: - (a) data + next - (b) data + next + prev - (c) միայն data - (d) 2 data

Q6

Stack-ը ինչ սկզբունքով է աշխատում: - (a) FIFO - (b) LIFO - (c) sorted - (d) random

Q7

Queue-ն ինչ սկզբունքով է աշխատում: - (a) LIFO - (b) FIFO - (c) sorted - (d) random

Q8

Stack-ի push/pop complexity: - (a) $O(n)$ - (b) $O(\log n)$ - (c) $O(1)$ - (d) $O(n^2)$

Q9

Որ DS-ն կա function call-երի համար (call stack): - (a) queue - (b) stack - (c) heap - (d) tree

Q10

BFS-ի համար որ DS-ն է օգտագործվում: - (a) stack - (b) queue - (c) heap - (d) trie

Q11

DFS-ի համար սովորաբար որ DS-ն է: - (a) queue - (b) stack / recursion - (c) heap - (d) hash

Q12

Hash table-ի միջին lookup complexity: - (a) $O(\log n)$ - (b) $O(1)$ average - (c) $O(n)$ always - (d) $O(n^2)$

Q13

Hash table-ի worst case complexity: - (a) $O(1)$ - (b) $O(\log n)$ - (c) $O(n)$ - (d) $O(n^2)$

Q14

Ի՞նչ է collision-ը hash table-ում: - (a) key ջնջում - (b) երկու key նույն bucket/hash - (c) memory leak - (d) sorted կարգ

Q15

Balanced BST-ի search complexity: - (a) $O(1)$ - (b) $O(\log n)$ - (c) $O(n)$ - (d) $O(n \log n)$

Q16

BST-ի inorder traversal-ը ինչ կարգ է տալիս: - (a) random - (b) sorted (աճ) - (c) reverse sorted - (d) level order

Q17

Unbalanced BST-ի worst case: - (a) $O(1)$ - (b) $O(\log n)$ - (c) $O(n)$ - (d) $O(n^2)$

Q18

C++ `std::map` -ը ինչ tree է օգտագործում: - (a) AVL - (b) Red-Black Tree - (c) B-Tree - (d) Trie

Q19

Max-heap-ում root-ը ինչ է: - (a) ամենափոքրը - (b) ամենամեծը - (c) միջինը - (d) random

Q20

Heap-ի insert (push) complexity: - (a) $O(1)$ - (b) $O(\log n)$ - (c) $O(n)$ - (d) $O(n \log n)$

Q21

Heap-ի top (peek) complexity: - (a) $O(1)$ - (b) $O(\log n)$ - (c) $O(n)$ - (d) $O(n \log n)$

Q22

C++ `priority_queue` -ի default-ով ինչ heap է: - (a) min-heap - (b) max-heap - (c) balanced - (d) sorted

Q23

Binary tree-ի inorder = ? - (a) Root Left Right - (b) Left Root Right - (c) Left Right Root - (d) Right Root Left

Q24

Preorder = ? - (a) Root Left Right - (b) Left Root Right - (c) Left Right Root - (d) level order

Q25

Postorder = ? - (a) Root Left Right - (b) Left Root Right - (c) Left Right Root - (d) Right Left Root

Q26

Level-order traversal-ը ինչ DS-ով է: - (a) stack - (b) queue - (c) heap - (d) recursion

Q27

Trie-ի search/insert complexity (L = բառի երկարություն): - (a) $O(1)$ - (b) $O(L)$ - (c) $O(n)$ - (d) $O(\log n)$

Q28

Trie-ն ինչի համար է ամենահարմար: - (a) sorting - (b) prefix search / autocomplete - (c) shortest path - (d) frequency

Q29

Graph-ի adjacency list-ի space complexity: - (a) $O(V^2)$ - (b) $O(V + E)$ - (c) $O(E^2)$ - (d) $O(1)$

Q30

Adjacency matrix-ի space complexity: - (a) $O(V + E)$ - (b) $O(V^2)$ - (c) $O(E)$ - (d) $O(1)$

Q31

BFS/DFS complexity (adjacency list): - (a) $O(V^2)$ - (b) $O(V + E)$ - (c) $O(V * E)$ - (d) $O(E^2)$

Q32

Array-ի մեջտեղում insert complexity: - (a) $O(1)$ - (b) $O(\log n)$ - (c) $O(n)$ - (d) $O(n^2)$

Q33

Linked list-ի head-ում insert: - (a) $O(1)$ - (b) $O(n)$ - (c) $O(\log n)$ - (d) $O(n^2)$

Q34

Որ DS-ն է ամենալավը frequency count-ի համար: - (a) array - (b) unordered_map (hash) - (c) stack - (d) queue

Q35

Որ DS-ն է պետք sorted կարգ + range query: - (a) unordered_map - (b) map / set (BST) - (c) stack - (d) queue

Q36

Top-K ամենամեծ element-ների համար: - (a) stack - (b) heap (priority_queue) - (c) queue - (d) trie

Q37

Որ traversal-ն է տալիս BST sorted: - (a) preorder - (b) inorder - (c) postorder - (d) level

Q38

Cache locality-ի տեսանկյունով որ DS-ն է ավելի արագ: - (a) linked list - (b) array/vector (contiguous) - (c) tree - (d) hash

Q39

Stack-ով ինչ problem է դասական լուծվում: - (a) shortest path - (b) balanced parentheses - (c) frequency - (d) sorting

Q40

Dijkstra-ի shortest path-ի համար որ DS-ն է օգտագործվում: - (a) stack - (b) min-heap (priority_queue) - (c) trie - (d) hash only

ՄԱՍ B — Open Questions (40)

Բաց հարցեր, փորձիր ամբողջական պատասխանել: Մոդելային պատասխանները՝ Answer Key B-ում.

1. Ի՞նչ է data structure-ը, ինչպես ես ընտրում որ մեկը օգտագործել:
2. Array vs Linked List — մանրամասն տարբերությունը:
3. Ինչո՞ւ array-ի access-ը $O(1)$ է, իսկ linked list-ինը $O(n)$:
4. Singly vs Doubly linked list:
5. Ինչպես ես reverse անում linked list (առանց կոդ, տրամաբանություն):
6. Stack-ը ինչ է, LIFO-ն բացատրի, ինչ problem-ներ է լուծում:
7. Queue-ն ինչ է, FIFO-ն բացատրի, ուր է օգտագործվում:

8. Stack vs Queue տարբերությունը:
 9. Ի՞նչ է circular queue / deque:
 10. Hash table ինչպես է աշխատում (hash function, bucket):
 11. Ի՞նչ է collision-ը, ինչպես է լուծվում (chaining, open addressing):
 12. Ինչո՞ւ hash table-ի worst case-ը $O(n)$ է:
 13. Ի՞նչ է load factor / rehashing:
 14. Tree vs Graph տարբերությունը:
 15. Binary tree vs Binary Search Tree:
 16. BST-ի inorder traversal ինչո՞ւ է sorted:
 17. Ինչո՞ւ BST-ն կարող է $O(n)$ դառնալ, ինչպես լուծել:
 18. Ի՞նչ են AVL / Red-Black Tree:
 19. Traversal-ների տեսակները (inorder/preorder/postorder/level):
 20. Ի՞նչ է heap-ը, max vs min heap:
 21. Heap-ի insert/extract ինչպես է աշխատում (heapify):
 22. Priority queue ինչի՞նչ համար է, ներսում ինչ է:
 23. Ի՞նչ է complete binary tree:
 24. Graph-ի ներկայացման ձևերը (adjacency list vs matrix):
 25. BFS vs DFS — երբ որ մեկը օգտագործել:
 26. Directed vs Undirected, Weighted vs Unweighted graph:
 27. Ի՞նչ է cycle-ը graph-ում, ինչպես է հայտնաբերվում:
 28. Ի՞նչ է topological sort:
 29. Trie-ն ինչ է, ինչի՞նչ համար է:
 30. Ի՞նչ է connected component:
 31. Երբ ես օգտագործում hash table vs BST:
 32. Երբ ես օգտագործում array vs linked list:
 33. Ի՞նչ է amortized complexity (vector push_back):
 34. Ի՞նչ է in-place algorithm:
 35. Ի՞նչ է stable data structure/sort:
 36. Ի՞նչ է space vs time tradeoff, բեր օրինակ:
 37. Ինչպես ես հայտնաբերում cycle linked list-ում (Floyd):
 38. Ի՞նչ է LRU cache, ինչ DS-երով է սարքվում:
 39. Stack-ով ինչպես ես implement անում queue (կամ հակառակը):
 40. Որ DS-ն ես ընտրում: fast lookup + sorted iteration երկուսի համար:
-

ՄԱՍ C — Tricky Code / Complexity (20)

C1

```
vector<int> v = {1,2,3};  
v.insert(v.begin(), 0);
```

Ի՞նչ complexity է այս insert-ը: - (a) $O(1)$ - (b) $O(n)$ (բոլոր element-ները սահում) - (c) $O(\log n)$ - (d) CE

C2

```
stack<int> st;  
st.push(1); st.push(2); st.push(3);  
st.pop();  
cout << st.top();
```

- (a) 1
- (b) 2
- (c) 3
- (d) CE

C3

```
queue<int> q;  
q.push(1); q.push(2); q.push(3);  
q.pop();  
cout << q.front();
```

- (a) 1
- (b) 2
- (c) 3
- (d) CE

C4

```
priority_queue<int> pq;  
pq.push(3); pq.push(1); pq.push(5);  
cout << pq.top();
```

- (a) 1
- (b) 3
- (c) 5
- (d) 9

C5

```
priority_queue<int, vector<int>, greater<int>> pq;  
pq.push(3); pq.push(1); pq.push(5);  
cout << pq.top();
```

- (a) 1
- (b) 3
- (c) 5
- (d) CE

C6

BST-ի inorder traversal, tree:

```
    5  
   / \  
  3   8  
 / \  
1  4
```

Ի՞նչ կարգ է տալվում inorder-ով: - (a) 5 3 8 1 4 - (b) 1 3 4 5 8 - (c) 1 4 3 8 5 - (d) 5 3 1 4 8

C7

Նույն tree-ի preorder: - (a) 5 3 1 4 8 - (b) 1 3 4 5 8 - (c) 1 4 3 8 5 - (d) 5 8 3 4 1

C8

```
unordered_map<int,int> m;  
m[5]++;  
cout << m[5];
```

- (a) 0
- (b) 1
- (c) undefined (garbage)
- (d) CE

C9

```
map<int,int> m;  
m[3]=1; m[1]=1; m[2]=1;  
for (auto& p : m) cout << p.first << " ";
```

- (a) 3 1 2
- (b) 1 2 3
- (c) random
- (d) CE

C10

```
unordered_map<int,int> m;  
m[3]=1; m[1]=1; m[2]=1;  
for (auto& p : m) cout << p.first << " ";
```

- (a) միշտ 1 2 3
- (b) միշտ 3 1 2
- (c) կարգ երաշխավորված չէ
- (d) CE

C11

Balanced BST-ում n element, search complexity: - (a) $O(1)$ - (b) $O(\log n)$ - (c) $O(n)$ - (d) $O(n \log n)$

C12

Sorted array-ի մեջ binary search:

```
vector<int> v = {1,3,5,7,9};  
binary_search(v.begin(), v.end(), 5);
```

- (a) $O(n)$
- (b) $O(\log n)$
- (c) $O(1)$
- (d) CE

C13

```
list<int> lst = {1,2,3};  
auto it = lst.begin();  
it + 2;
```

- (a) OK -> 3
- (b) compile error (list random access չունի)
- (c) $O(n)$
- (d) undefined

C14

```
vector<int> v = {1,2,3};  
auto it = v.begin();  
it + 2;  
cout << *it;
```

- (a) compile error
- (b) 1 (it ինքը չի փոխվում)
- (c) 3
- (d) undefined

C15

Heap-ի (priority_queue) build heap complexity n element-ից: - (a) $O(n)$ - (b) $O(n \log n)$ - (c) $O(\log n)$ - (d) $O(n^2)$

C16

Graph adjacency matrix, edge (u,v) կա թե չէ ստուգում: - (a) $O(V)$ - (b) $O(1)$ - (c) $O(E)$ - (d) $O(V+E)$

C17

```
stack<int> st;  
cout << st.top();    // դատարկ stack
```

- (a) 0
- (b) undefined behavior (empty stack top)
- (c) CE
- (d) -1

C18

Linked list-ի մեջտեղին (middle) հասնելու complexity: - (a) $O(1)$ - (b) $O(n)$ - (c) $O(\log n)$ - (d) $O(n^2)$

C19

```
vector<int> v;  
v.reserve(100);  
cout << v.size() << " " << (v.capacity() >= 100);
```

- (a) 100 1
- (b) 0 1
- (c) 100 0
- (d) 0 0

C20

Trie-ում "apple" բառի search complexity: - (a) $O(1)$ - (b) $O(5) = O(L)$ (բառի երկարություն) - (c) $O(n)$ - (d) $O(\log n)$

ANSWER KEY A (Multiple Choice)

1-a · 2-b · 3-b · 4-b · 5-b · 6-b · 7-b · 8-c · 9-b · 10-b 11-b · 12-b · 13-c · 14-b · 15-b · 16-b · 17-c · 18-b ·
19-b · 20-b 21-a · 22-b · 23-b · 24-a · 25-c · 26-b · 27-b · 28-b · 29-b · 30-b 31-b · 32-c · 33-a · 34-b · 35-b
· 36-b · 37-b · 38-b · 39-b · 40-b

ANSWER KEY B (համառոտ)

1. DS-ը տվյալների կազմակերպման ձև է արդյունավետ մուտք/փոխում/որոնում-ի համար. Ընտրում ես ըստ՝ ինչ գործողություններ են հաճախակի, complexity, memory. Ճիշտ DS -> արագ algorithm:
2. Array՝ contiguous memory, $O(1)$ index access, $O(n)$ insert/delete մեջտեղում, cache-friendly. Linked list՝ scattered, $O(n)$ access, $O(1)$ insert/delete (pointer-ով), ավելի շատ memory (pointer-ներ):
3. Array-ի element-ները կողք-կողքի են -> հասցե = $\text{base} + i * \text{size}$ (ուղիղ հաշվարկ). Linked list-ի node-ները scattered են -> պետք է քայլ առ քայլ գնալ head-ից (index չկա):
4. Singly՝ ամեն node data + next (միայն առաջ). Doubly՝ data + next + prev (առաջ ու հետ):
5. Երեք pointer (prev, curr, next); ամեն քայլ curr-ի next-ը ուղղում prev-ին, առաջ շարժվում. վերջում prev-ը նոր head է:
6. Stack՝ LIFO (Last In First Out), վերջին ավելացրածը առաջ հանվում. push/pop/top $O(1)$. Problem-ներ՝ parentheses, undo/redo, DFS, function call stack, expression eval:
7. Queue՝ FIFO (First In First Out), առաջ ավելացրածը առաջ հանվում. push (back)/pop (front) $O(1)$. Օգտագործում՝ BFS, scheduling, buffering:
8. Stack LIFO (մեկ ծայրից), Queue FIFO (երկու ծայրից՝ back-ում add, front-ից remove):
9. Circular queue՝ array-based queue, որ վերջից վերադառնում սկիզբ (memory reuse). Deque՝ double-ended queue, երկու ծայրից push/pop $O(1)$:
10. hash(key) -> bucket index -> element պահվում այդ bucket-ում. Lookup՝ hash հաշվել -> bucket գտնել -> $O(1)$ average:
11. Collision՝ երկու key նույն bucket. Լուծում՝ chaining (linked list bucket-ում) կամ open addressing (հաջորդ ազատ bucket):
12. Շատ collision կամ վատ hash function -> bucket-ի list երկարում -> lookup $O(n)$ դառնում. Կամ rehashing-ի ժամանակ:
13. Load factor = $\text{size}/\text{buckets}$. Գերազանցելիս (>1) rehash՝ buckets x2, բոլոր element վերաբաշխում ($O(n)$ այդ պահին):
14. Tree՝ hierarchical, connected, acyclic, n node -> n-1 edge, մեկ root. Graph՝ vertices + edges, կարող է cycle, disconnected, weighted լինել:
15. Binary tree՝ ամեն node ≤ 2 child, կանոն չկա. BST՝ $\text{left} < \text{root} < \text{right}$ (ամեն node-ի համար) -> sorted, search $O(\log n)$ balanced:
16. Inorder՝ Left-Root-Right. BST-ի կանոնը ($\text{left} < \text{root} < \text{right}$) ապահովում է, որ սա տալիս է աճ sorted կարգ:

- 17.** Sorted տվյալ insert -> tree դառնում linked list (մեկ կողմ) -> height $O(n)$ -> search $O(n)$.
Լուծում՝ self-balancing (AVL, Red-Black Tree):
- 18.** AVL՝ խիստ balanced BST (height diff ≤ 1). Red-Black՝ մոտավոր balanced, ավելի քիչ rotation insert-ի ժամանակ (std::map/set-ը RB-tree է):
- 19.** inorder (L-Root-R), preorder (Root-L-R), postorder (L-R-Root), level-order (BFS, queue-ով, մակ-մակ):
- 20.** Heap՝ complete binary tree. Max-heap՝ parent $\geq$ child (root max); min-heap՝ parent $\leq$ child (root min). Սովորաբար array-ով է ներկայացվում:
- 21.** Insert՝ ավելացնում վերջում, հետո "bubble up" (parent-ի հետ փոխանակում մինչև կանոն) -> $O(\log n)$. Extract՝ root հանում, վերջին element վեր, "bubble down" (heapify) -> $O(\log n)$:
- 22.** Priority queue՝ միշտ ամենամեծ/փոքր element-ը տալիս. ներսում՝ heap. push/pop $O(\log n)$, top $O(1)$. C++ default max-heap:
- 23.** Complete binary tree՝ բոլոր level-ները լցված, բացի վերջին-ից որ լցվում է ձախից աջ. Heap-ը complete binary tree է:
- 24.** Adjacency list՝ ամեն vertex-ի հարևանների ցուցակ, $O(V+E)$ space, sparse-ի համար. Adjacency matrix՝ $V \times V$ table, $O(V^2)$, edge check $O(1)$, dense-ի համար:
- 25.** BFS՝ queue, level-by-level, shortest path unweighted, ողղային. DFS՝ stack/recursion, խորը, cycle/topo sort/components. Երկուսն էլ $O(V+E)$:
- 26.** Directed՝ edge մեկ ուղղությամբ ($A \rightarrow B$); undirected՝ երկու. Weighted՝ edge-ը ունի կշիռ (cost); unweighted՝ բոլորը հավասար:
- 27.** Cycle՝ ուղի որ վերադառնում է սկիզբ. Undirected՝ DFS + parent (visited հարևան որ parent չէ -> cycle). Directed՝ recursion stack-ում visited node:
- 28.** Topological sort՝ DAG-ի vertex-ների գծային կարգ, որտեղ ամեն edge $u \rightarrow v$ -ի համար u -ն գալիս է v -ից առաջ. Kahn's (in-degree + BFS) կամ DFS:
- 29.** Trie՝ prefix tree string-երի համար, ամեն node մեկ char. insert/search/prefix $O(L)$. Autocomplete, spell check, dictionary:
- 30.** Connected component՝ graph-ի ենթախումբ, որտեղ ամեն vertex հասանելի է մյուսից. DFS/BFS-ով հաշվում են քանակը:
- 31.** Hash (unordered_map)՝ $O(1)$ avg lookup, կարգը պետք չէ, frequency/dedup. BST (map)՝ $O(\log n)$, sorted, range query / lower_bound պետք է:
- 32.** Array՝ random access, index, sorting, չափը հայտնի. Linked list՝ հաճախակի insert/delete, չափը անհայտ, iterator stability:
- 33.** Amortized complexity՝ միջին արժեքը շատ գործողությունների վրա. vector push_back՝ սովորաբար $O(1)$, reallocation-ը $O(n)$, բայց $x2$ growth -> միջինը amortized $O(1)$:
- 34.** In-place՝ algorithm որ չի օգտագործում լրացուցիչ (կամ $O(1)$) memory, փոխում ուղիղ input-ը (օրինակ՝ reverse, bubble sort):

35. Stable՝ պահպանում է հավասար element-ների սկզբնական կարգը (sort-ի դեպքում). Stable sort՝ merge, insertion; unstable՝ quick, heap:

36. Space-time tradeoff՝ ավելի շատ memory -> ավելի արագ (օրինակ hash table $O(1)$ lookup բայց $O(n)$ memory); կամ քիչ memory -> ավելի դանդաղ:

37. Floyd's Tortoise & Hare՝ երկու pointer (slow 1 քայլ, fast 2 քայլ). Cycle-ի դեպքում հանդիպում են. $O(n)$ time, $O(1)$ space:

38. LRU cache՝ hash map + doubly linked list. Map՝ $O(1)$ lookup; list՝ կարգ (վերջերից հին). get/put $O(1)$:

39. Queue երկու stack-ով՝ մեկ in-stack (push), մեկ out-stack (pop); out դատարկ լինելիս in-ը շարժում out (reverse). Amortized $O(1)$:

40. map (BST) -> fast ($O(\log n)$) lookup + sorted iteration; կամ hash map + sorted structure. Եթե երկուսն էլ կրիտիկ՝ map (sorted iteration hash-ը չունի):

ANSWER KEY C (Tricky Code)

C1-b · C2-b · C3-b · C4-c · C5-a · C6-b · C7-a · C8-b · C9-b · C10-c · C11-b · C12-b · C13-b · C14-b · C15-a · C16-b · C17-b · C18-b · C19-b · C20-b

Բացատրություններ (C)

C1 begin-ում insert -> բոլոր element-ներ մեկ քայլ աջ -> $O(n)$: **C2** push 1,2,3 -> pop (3 հանվում) -> top = 2: **C3** push 1,2,3 -> pop (1 հանվում, FIFO) -> front = 2: **C4** default max-heap -> top = 5 (ամենամեծը): **C5** greater -> min-heap -> top = 1: **C6** inorder (L-Root-R) BST -> sorted -> 1 3 4 5 8: **C7** preorder (Root-L-R) -> 5 3 1 4 8: **C8** m[5]++ -> չկա key -> ստեղծում 0-ով, ++ -> 1: **C9** map sorted -> 1 2 3: **C10** unordered_map -> կարգ երաշխավորված չէ: **C11** balanced BST search -> $O(\log n)$: **C12** binary search sorted-ում -> $O(\log n)$: **C13** list random access չունի -> it+2 compile error: **C14** it+2-ը նոր iterator է ստեղծում բայց չի վերագրվում it-ին -> it մնում begin -> it=1: **C15** build heap n element-ից -> $O(n)$ (նչ $O(n \log n)$): **C16** adjacency matrix edge check -> mat[u][v] -> $O(1)$: **C17** empty stack-ի top() -> undefined behavior: **C18** linked list middle-ի հասնել -> $O(n)$ (քայլ առ քայլ): **C19** reserve՝ size=0, capacity>=100 -> "0 1": **C20*** trie "apple" search -> $O(L) = O(5)$:

Ընդհանուր կանոններ (հիշիր interview-ին)

Array: access $O(1)$, insert/delete mid $O(n)$, contiguous
List: access $O(n)$, insert/delete $O(1)$ w/ pointer, scattered
Stack: LIFO, push/pop/top $O(1)$
Queue: FIFO, push/pop/front $O(1)$
Hash: $O(1)$ avg, $O(n)$ worst, unsorted
BST: $O(\log n)$ balanced / $O(n)$ unbalanced, inorder=sorted
Heap: top $O(1)$, push/pop $O(\log n)$, build $O(n)$, default max
Trie: $O(L)$ operations, prefix
Graph: BFS(queue)/DFS(stack) $O(V+E)$, adj list (sparse)/matrix (dense $O(1)$ edge)

TRAVERSAL: inorder=LRootR (BST sorted) | preorder=RootLR | postorder=LRRRoot | level=BFS
map=sorted ($O(\log n)$) | unordered_map=hash ($O(1)$ avg, no order)
empty stack/queue top/front -> UB